

SURVEY

Extensive Review of Threat Models for DevSecOps

NAGASUNDARI S^{ID}, (Member, IEEE), PRATHAM MANJA^{ID}, PRIYANSH MATHUR^{ID},
AND PRASAD B. HONNAVALI^{ID}, (Member, IEEE)

Department of CSE, Center for Information Security, Forensics and Cyber Resilience, PES University, Bengaluru 560085, India

Corresponding author: Nagasundari S (nagasundaris@pes.edu)

This work was supported by PES University.

ABSTRACT Amidst the rising tide of cyber threats, the software industry is gravitating toward the widespread adoption of DevSecOps as a crucial defense strategy. In response to the evolving application development needs, DevSecOps enables developers and enterprises to seamlessly integrate security and, fortify digital realms against modern cybersecurity challenges. This study attempts to identify and review various threat models that can be combined with components of DevSecOps, such as its principles, methodology, tools, and workflows. Underscoring the necessity of early security considerations in the software development life cycle, the discussion then delves into the details of adaptable strategies and best practices to mitigate potential threats proactively, while also emphasizing the pivotal role of monitoring and incident response in the prompt detection and mitigation of potential risks. Moreover, this study focuses on identifying the challenges associated with adopting traditional threat models in the contemporary landscape of multi-cloud architecture, a prevalent paradigm in current technological landscapes. Recognizing the intricacies of securing applications across diverse cloud environments, this scholarly work proposes a layered framework for multi-cloud, which can be adopted by practitioners to address the security challenges of the multi-cloud environment. Hence, this investigation furnishes valuable insights into the requirements, workflow and threat models of DevSecOps in single-cloud and emerging multi-cloud environment.

INDEX TERMS DevOps, DevSecOps, threat modeling, application security, vulnerability assessment, layered architecture.

I. INTRODUCTION

Threats to system security are a major concern for organizations today. One of the notable attacks that occurred in March 2023 involved the injection of malicious code into VoIP (Voice over Internet Protocol) software, specifically the 3CX Desktop Application. The attackers succeeded by compromising the software build pipeline. This incident highlights the vulnerability of software supply chains and the need for improved detection methods beyond typical security testing. To address this oversight, organizations need tools that track changes in software code to identify suspicious modifications. Kraemer [1] revealed that individuals operating under tight schedules and experiencing an uneven distribution of workloads inadvertently introduce security vulnerabilities

into software systems. According to the aforementioned researchers, the primary source of vulnerabilities is the lack of security understanding in the DevOps teams. This minimizes the detection of vulnerabilities in the application by lowering the quality of security evaluation. Furthermore, cybercrime has been on an ever-increasing trend in recent years. A recent study reported 3122 data breaches that impacted an estimate of 353,027,892 individuals by 2023. This indicates a 72% increase in such events compared to incidents reported in 2021 [2].

According to Cybersecurity insiders [3], as depicted in Figure 1, 27% of customizations hold DevOps accountable for security remediation. DevOps is a prominent methodology in the software industry that attempts to automate manual efforts. Businesses are now not only investing in shorter development cycles and speedy delivery but are also trying to reduce their threat surface. A major focus is now on

The associate editor coordinating the review of this manuscript and approving it for publication was Padmanabh Thakur^{ID}.



FIGURE 1. Importance of DevSecOps.

integrating security early in the pipeline rather than wrapping up the final product.

According to a study by Gartner [4], Chief Information Security Officers (CISOs) need to enhance their assessment practices by implementing Continuous Threat Exposure Management (CTEM) programs to better understand their risk exposure. Gartner also predicted that by 2026, organizations that are prioritizing their security investments based on a CTEM program would have two-thirds fewer breaches, protecting their assets from any external assault. Data is a critical asset that must be protected at all times. This can be achieved by creating a responsive ecosystem that promotes sustainable and well-balanced cyber-security programs to safeguard processes, technology, tools, and infrastructure.

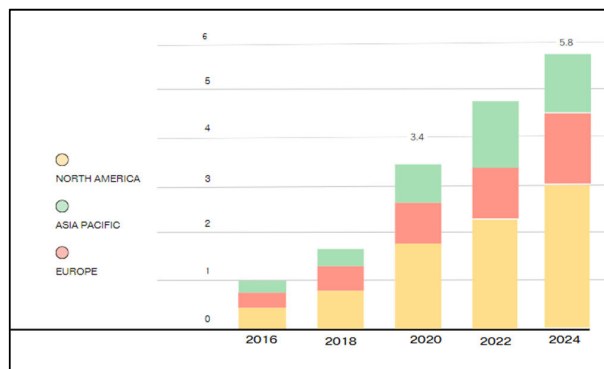


FIGURE 2. DevSecOps market cap by region in billion dollars.

Figure 2, infers that the annual budget of the regions is on a steady rise and focuses on the DevSecOps market [5]. DevSecOps, in its higher form, is merely shifting the security practices left in the DevOps pipeline. Integrating security would help products to delivered rapidly, efficiently, and securely. Currently, the majority of the annual budget is dedicated to risk assessment and analysis measures to avoid disrupting Business Continuity. A study by Gartner suggests that “securing the software delivery pipeline is as important as securing the software that is delivered.” Traditionally, security is provided after a product leaves its development and operation pipeline. However, the scope of an attack can be way before it arrives at the market and is open to public traffic. The more infected the code becomes at an early stage, the more challenging it will be to secure in the future. Therefore, regardless of the size, budget, or global presence of software,

security is now integrated from the root of the pipeline, which is generally where the development of the product starts.

One of the key findings of the survey conducted by Contrast Security [6] was that 79% of the average applications in development have more than 20 vulnerabilities, and 99% of the average application in production has more than four vulnerabilities. These alarming numbers are indicators of the number of vulnerabilities that crept into the software well before delivery. This supports the fact that there is a chance that the pipeline will be infected, which in turn weakens the software artifacts that reside in it. This survey also pointed out that each vulnerability consumes more than 4 hours for a developer, which leads 55% of the developers to skip the security scans to meet deadlines. Surprisingly, this poses a significant setback to the cybersecurity measures safeguarding software. Vulnerability testing is a process that identifies and diagnoses weakness by testing towards the end of the Software Development Life Cycle (SDLC).

The higher the severity of vulnerability, the longer it takes to eliminate it. Developers now are not required to just write code that runs a business module but to write it in a secure manner. Upon failure, the security debugging time of an application becomes significant. Corporations sometimes miss some security processes and techniques such as Static Application Security Testing (SAST), Interactive Application Security Testing (IAST) and Dynamic Application Security Testing (DAST) to meet hard deadlines. Although the project looks ready on the outside, it is fragile and brittle on the core structure. It is just like saying that we wrap a golden foil on an apple to make it Golden Apple. Sooner or later, the threats slowly start swelling in the product, and as time passes, they will create a huge crater allowing illegal and unsolicited entry into critical and sensitive parts of one or more components.

The Global Market for DevSecOps is expected to grow significantly and reach a substantial size by 2030. The forecast indicates a Compound Annual Growth Rate (CAGR) of 27.7% for this market. In 2022, the DevSecOps market in the United States was estimated to be valued at US\$1.9 billion. China, which is the world’s second-largest economy, is projected to achieve a market size of US\$7.9 billion by 2030, representing a CAGR of 33.5% from 2022 to 2030. Several other geographic markets are also noteworthy. Japan is expected to grow at a CAGR of 21.5% from 2022 to 2030, whereas Canada is forecast to experience a growth rate of 22.9% during the same period. In Europe, Germany is expected to grow at an approximate CAGR of 23.8%. These forecasts demonstrate the increasing significance and adoption of DevSecOps practices and technologies in various regions worldwide. These high growth rates indicate a growing demand for DevSecOps solutions and services in the market [7].

With the importance given to the adoption of security in the DevOps model, as seen by the budget spent, risk mitigation and vulnerability assessment are now key players in the creation of secure and malware-free software that aims to harness the power of cyber-security to strengthen the

application from inside thereby increasing client satisfaction and convenience. Automation of these security scans is also a trending topic that encompasses the major processes and tools being integrated into a development system altogether. Automation Of Security Scans (AOSS) will not only make the pipelines run faster but also eliminate any manual errors that may be missed by the Security Analyst. AOSS can also be integrated with state-of-the-art AI tools, which may provide a visualization of the vulnerabilities found in a module along with its severity level. AOSS reduces the magnitude of the severity of the whirlpool from escalating to an unmanageable level in the future by addressing and eliminating it early on.

With security being a paramount consideration in cloud-native software development, this research paper centers on the following key aspects:

- 1) Highlights the transformations and adaptations of DevSecOps beyond DevOps ideology
- 2) Review of various threat models relevant to DevOps-driven product development and their utilization to address potential threats and attacks in a cloud-native DevSecOps pipeline
- 3) Proposes a layered framework that could be implemented further in order to address the challenge of adopting traditional threat models in a multi-cloud environment.

This paper is structured into eight meticulously crafted sections, each contributing to a comprehensive exploration of DevSecOps. It begins by laying the foundation with a background section that traces the evolution from DevOps to the more robust DevSecOps paradigm, followed by an extensive review of existing literature in this domain, encompassing insightful surveys and their significant findings. Section IV delves into the indispensable value of Risk Assessment within IT organizations. Section V presents a spotlight on the intricacies of threat modeling. It encompasses an in-depth exploration of three distinct service models, offering a tailored approach to threat modeling by dissecting popular attacks within each model and presenting potential mitigation strategies. Section VI highlights the challenges in adopting traditional threat models for multi-cloud architecture (MCA) while proposing a layered architecture to address these challenges using new technologies and approaches. Section VII illuminates the diverse Applications and Use Cases of DevSecOps across various domains.

II. BACKGROUND

In the early 2000s, software development rose steeply, establishing a solid foundation in the Information Technology (IT) industry. Popular dynamic applications have been developed using languages such as C, C++, RubyOnRails and more. Tireless efforts in this field have focused on developing an approach for streamlining the workflow of software development. Few of the top organizations at that time used proprietary methods. Eventually, software companies felt the need to establish a common ground for development. People started facing issues in collaborating with other developers

because of the incompatibility in their practices. Not only did this affect the culture but also other technical practices followed by them.

A. INTRODUCTION TO DEVOPS

For software applications to be released in the open market, developers must ensure that all modules comprising the application integrate seamlessly. A team of engineers is typically deployed to develop a single project. Teams are assigned to a particular job to ensure that it is completed within the stipulated time. These engineers complete their tasks independently and push them to an archive upon completion. This process is identical for all engineers at all levels and domains. After the successful completion of the task and the updated status of the archive, the application is further tested. Testing can be simply defined as the process of discovering, analyzing, and resolving any bugs or issues that disrupt the normal behavior of the software. When the project manager identifies any gaps in the product delivery, and contacts the development team, the team must follow the established process to address the issues. This involves tracing the problem back to a specific module, determining the module's domain, contacting the responsible team leader, and notifying them accordingly. After some time, the project manager receives a response from the domain's team leader and pushes the modified code to the archive. This process is repeated if further errors are detected. This implies that software development is a tedious process involving the development and testing phase of the Software Development Lifecycle. Both of these phases could also involve the collaboration of multiple teams. This entire process highlights two key takeaways:

1. The need for a framework that incorporates processes and best practices to enhance the efficiency of software product delivery
2. The importance of automating repetitive tasks to accelerate workflows and create more room for innovation.

Apart from design, development and testing, there is a need for another team known as the Operations Team. This operations team's primary role is to facilitate resource provisioning for developers. This team assumes responsibility for managing the organization's infrastructure. The infrastructure can include routers, switches, Internet gateways, servers, etc. The Operations Team is dedicated to provide computational resources which aids the developers in addressing intricate and extensive problems. Consequently, this engenders a new type of dependency on the Operations Team, as they are relied upon to promptly and consistently fulfill requests from the development team. When multiple urgent requests from different departments arise simultaneously, bottlenecks can occur, hindering the speed of service delivery. Considering resource-intensive applications that consume a large number of computational services, starvation and infinitely long blocking were introduced to other applications in the queue. It was around 2007-2008, when a new buzzword called DevOps was born in the IT industry.

In its fundamental essence, DevOps is a fusion of two pivotal domains: Development and Operations. DevOps encapsulates a collection of optimal practices, tools, and a cultural philosophy designed to automate and seamlessly integrate processes between software development and IT teams. Emphasizing cross-team collaboration and communication, it seeks to forge a shared understanding among individuals from diverse domains and technological backgrounds. It is an IT paradigm for agile development and efficient maintenance that builds software in a more organized and structured manner. DevOps also emphasizes aligning development and operations to boost quality, efficiency, and productivity.



FIGURE 3. DevOps process.

After several years, DevOps has brought the development and operations teams close together, despite the fact that they were binned. These two teams get together and software engineers work on an entire application lifecycle ranging from development, testing, deployment, operations and various other multidisciplinary activities. DevOps is a software ideology that aims to develop a framework and deliver tools for task automation. As previously discussed, these two deliveries were important in the early 2000s. Thus, DevOps was an essential and necessary change that Software Engineers embraced and welcomed with open hands as they had learned from their past experience that the adoption of this comprehensive process will not only help in cross-team management but will also act as a Process Phase tracker to help monitor their progress.

Pertaining to the perpetual character of DevOps, professionals commonly illustrate the DevOps Lifecycle as an infinite loop. This visual representation underscores the interconnectedness of DevOps phases, emphasizing the seamless relationship between them. The core of DevOps lies in continuous collaboration and incremental improvements to satisfy clients and ease management activities performed throughout the process. DevOps consists of eight phases, as illustrated in Figure 3 and described below:

Discover: Building software requires teamwork. Teams should conduct workshops to plan, organize and analyze

ideas. These ideas should focus on achieving goals and fulfilling customer requirements. Agile is widely used for this purpose. During the Discover phase, a DevOps team does research and specifies the scope of a project. It entails actions such as user research, goal setting, and defining the steps to success.

Plan: DevOps teams should be comfortable adopting agile practices. Agile is an iterative approach to software development and project management that aims to deliver small pieces of work incrementally.

Build: This phase uses tools to build software components such as libraries and modules. Git is an open-source and common platform for version control which allows developers to work on branches efficiently. This allows one to recover previous builds by tracking their repository history. The Build phase is when developers begin implementing additions, updates, bug fixes, or new features. Project managers and product owners evaluate the input obtained from the previous development cycle to identify the internal or external backlog items, which will deliver the most value to the end customer. If the team implements an agile development approach, then sprints are organized and specific problems are allocated to different team members.

Test: Continuous Integration (CI) ensures that it merges the developer's code in a shared repository with the master's (main) repository. This not only provides reassurance to developers regarding their work but also improves their predictability about the software. DevOps approaches necessitate a rapid release of software, with as many deployments per day feasible. Teams must be able to test the code rapidly and frequently in order to satisfy these criteria. The Test Phase will decide whether the build is prepared to advance to the further steps or it needs to be delayed and corrected.

Operate: The Operations team manages the complete delivery of IT services to users. This includes practices involving the design, configuration, and maintenance of all IT infrastructure that supports an organization's services. The operations team uses automated tools for configuration management, scalability, and load balancing after the DevOps software has gone live in production.

Observe: The Operations team quickly identifies and resolves issues that can slow productivity and speed of deployment. In most areas, automated checks are used to analyze and notify teams about the changes, failures, and high-alert risks immediately, so that the application continues its execution without a break. After deploying and stabilizing an application in a production environment, it becomes imperative to implement additional safety measures. Businesses are required to conduct continuous monitoring of the live application, employing automated security checks and security monitoring loops to detect and respond to any potential attacks or breaches. A crucial component in this defense mechanism is Runtime Application Self-Protection (RASP), which swiftly and autonomously identifies and mitigates inbound security threats. Operating as a reverse proxy, RASP acts as a vigilant guardian, proactively adapting

the software to specific circumstances without requiring user intervention

Continuous feedback: DevOps teams should generate reports for each release and share feedback before the next release. This feedback integrated with newer releases helps to improve the standard of the software. It also aligns with the agile principle of preparing the software so that it can act upon frequent changes. The result of each release on the basis of user experience is assessed during the continuous feedback phase of the DevOps lifecycle, and the team is informed of the results to enhance subsequent releases. There are two ways to collect feedback: structured and unstructured feedback. Focus groups, surveys, and questionnaires are used to implement the structured technique. Unstructured comments are gathered through social media sites such as Twitter, and Facebook. Similar to how consumers review mobile applications on the Play Store, users contribute to the DevOps process by sharing their feedback.

B. TRANSITION FROM DEVOPS TO DEVSECOPS

Although DevOps played a key role in incorporating major changes in the software lifecycle, it neglected a major aspect of security as one of its phases, despite the increasing frequency of information security incidents and growing emphasis on security by major enterprises [8]. To address this, DevSecOps has emerged as a new approach that integrates security into the development, operations, and maintenance processes. It integrates DevOps with security practices [9], representing the natural progression of software development towards embedding security at every step of the development lifecycle. The cultural transformation instigated by DevSecOps establishes security as a collective responsibility for all participants in the software development process. [10]. DevSecOps comprises three terms: Development, Security, and Operations, and aims to combine cyber security practices with SDLC to seal off security issues that may creep in later on. As industries experience higher threats from malicious attacks, they have realized that providing an extra layer of security to the DevOps lifecycle can significantly reduce the attack rate. To securely continue business expansion, corporations must include security scans from the beginning of the pipeline. Rapidly assessing security needs becomes more challenging in DevSecOps-driven projects, as automated processes operate at high speed, making it difficult to fully validate security requirements only before deploying software in production [11].

A survey by Contrast Security, reported that 42% of the companies that suffered a breach attributed the cause to a known but unpatched vulnerability. Cyber Security is increasingly a priority for leaders such as the board of directors and C-suites. Verizon [6] found that 43% of data breaches in 2019 were a result of web-application vulnerability, which has doubled since 2018. One of the survey respondents also said that creating a balance and synergy between the security and development teams is the easiest and most economical

way to achieve software security. Figure 4 provides a comparison between the number of vulnerabilities found in an application during its development and deployment. One could clearly observe that securing applications from the risk of an attack is vital. As said by John P Mello Jr [7] in “The State of Application Security Testing: The Shift is on to Secure Code,” with the software development ground shifting, it’s time for application security teams to get a move on from application security to securing code throughout the software development life cycle. Security teams now need to join development teams to foster strong relations and overcome this gap by working in resonance. This is a “left shift” of security practices, which means that security inspection and scans are not performed on a product that is just waiting in the production environment to get deployed but is a process that must be interleaved with DevOps. In the multi-cloud era, businesses are facing critical challenges, with security being a top priority. As data and applications are spread across different cloud providers, maintaining a strong and uniform security stance becomes a formidable challenge. The DevSecOps concept has emerged prominently as a solution for addressing various needs in this context.

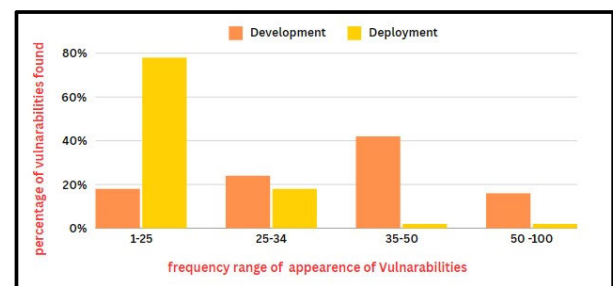


FIGURE 4. Vulnerabilities found in development vs deployment.

III. REVIEW OF EXISTING LITERATURE

This section discusses the existing work on DevOps security and attack models defined for various cloud service models including Iaas, Paas, and Saas.

One of the prominent studies by Sun et al. [12] described the structure of an efficient and accurate security test pipeline to identify security vulnerabilities and solutions to these vulnerabilities. The aforementioned authors outlined the specifications, that a security test pipeline must meet. The features of this security test pipeline system include centralized and unified security test management, regular and automatic code security testing, unified user management and authority management. To satisfy security and confidentiality compliance requirements, a comprehensive rule management function was realized using a test pipeline built upon open-source software tools which ensures a low-cost system to resolve code security and quality issues. McZara et al. [13] analyzed micro-service interdependency to model Interdependencies using links that connect one microservice to another. These links were nothing but an interface through which

one microservice was accessible to another. DevSecOps was utilized to monitor microservice outages, leading to the discovery of an alternative path that enabled fault tolerance and ensured business continuity as a recovery mechanism for the application. The author Hasan Yasar [14] examined the possibility of finding information security risks differently at different stages of the DevOps pipeline. Development teams often miss the issues to be addressed as they rely heavily on configuration management and automation. Agung and Kabetta [10] considered agile development as a modern method that emphasizes flexibility and spontaneity; however, this approach comes with greater risks and issues in the build and deployment phases that drain most of the time to get fixed leading to a delay in the release of the software. To address these challenges in agile development, the authors considered building a DevSecOps pipeline integrating static and dynamic testing in an SDLC environment, using the open-source Git-Lab for setting up a CI/CD pipeline and tools such as Njsscan and OWASP-ZAP for static and dynamic testing functions.

The Study performed by Bajpai and Lewis [15] describes the consequences of misconfiguration errors that malicious threats seek to exploit. This has been aggravated when the organizations consider development and security teams to be independent. The above researchers also presented areas that are more threat prone and solutions for monitoring these threats, including automated security assessments and dependency management. Finally, they stressed the monitoring of CI/CD pipelines to detect the possibility of involving new attack vectors and threat surfaces. Extending the above work, Mahboob et al. [16] used a Kubernetes Environment to create a CI/CD pipeline. They then used Asylo to perform security scans and Tekton to automate the process of creating a Docker file whenever there was a change in the branch (when committed). Emphasis was placed on using a Trusted Execution Environment (TEE) to isolate and secure native developments to harden security strength. A study carried out by Ramaj and Xhesika [17] discussed security that can be integrated into the DevOps environment into the development life cycle to address complex security threats and loose security. It also depicted the process of identifying vulnerabilities until the production stage, where debugging the application for threats takes a long time. Continuous Risk assessment is also valuable and should be a part of SDLC. An Investigation conducted by Aljohan and Alqahtani [18] described the consequences of the lack of empowerment of automated security testing tools. In addition, knowledge of when and where to add security automation to a DevOps pipeline is a major problem that must be addressed during software development. Some of the challenges encountered are automated scanning, the total time elapsed, and false-positive results. Finally, the authors developed an architecture that enabled automated security scans and vulnerability detection. Ibrahim et al. [19] explained the security model required for infrastructure as code (IaC) over the cloud, emphasizing the need for IaC in setting up infrastructure such as servers, load balancers, and databases over the cloud environment.

The main objective involved overcoming the challenges of setting up a completely secured and fully automated security pipeline in IaC. Finally, the aforementioned authors had set up a complete security pipeline using Terraform and Amazon S3 software.

The authors Sojan et al. [20] investigated the demands of automated testing stages in cloud-native environments. As these cloud platforms have a dynamic infrastructure, managing their security requirements becomes difficult. These authors mainly focused on the Infrastructure-as-a-Service (IaaS)-based cloud environment, thus developing an automated monitoring solution for cloud-native DevSecOps using open-source tools and software. The monitoring stage was divided into infrastructure and application monitoring using a microservice architecture. Therefore, the authors have developed a novel solution for addressing the gap in cloud monitoring applications is essential.

Taking the previous analysis further, Diaz et al. [21] developed a self-service cybersecurity monitoring system, further describing the fast and continuous feedback from operation to development using the SPEM model. Finally, a formalized DevOps cycle was created to build a self-service security pipeline so that teams can configure their monitoring services in accordance with their security criteria. Thus, allowing the teams to share responsibility in releasing secure software, which is key to enabling DevSecOps culture. Caraturan and Goya [22] analyzed the major financial challenges involved in adapting DevSecOps to a cloud-native platform. It is concluded that it is essential to have a vision of identifying the applications that are appropriate for the cloud environment and those are possible for migration. The implementation can be conducted in three stages: fundamental, repeatable, and scalable. These stages are fully automated, with indicators and feedback systematically collected and analyzed. Cankar et al. [23] developed software as a proof of concept that can perform both SAST and DAST on IaC scripts. They used VAT, which works on rule-based matching of scripts for static analysis, and LOMOS, which is an artificial intelligence (AI) tool for dynamic analysis. Static analysis uncovers design issues and performs component tests ranked from the most severe at the top and proceed in decreasing order. LOMOS uses self-supervised NLP techniques to automate the process by checking logs to find differences and uncover any threats. An investigation carried out by Pecka, and Nicholas Stephen [24] aims to prove that inside attackers can exploit the DevSecOps pipeline to transform secure software into insecure software. Kubernetes was used as the deployment environment in this study. Their workflow is as follows: Code is pushed to GitHub, where a webhook listener from Jenkins is triggered and submits the code with the changes. After the pipeline finishes, the end product is a Docker Container that is pushed to DockerHub and Jenkins records this on K8 logs. Then K8 uses the docker commands to retrieve the container for further activities. This paper also highlights the different points of attack that can be performed on Jenkins, and Docker.

IV. RISK ASSESSMENT

In the 20th century, organizations operated locally, serving clients who would visit their offices for direct interactions. Having a global presence before the dawn of the Internet was only possible via the branches of the company dispersed across the country. This implies that their presence was directly proportional to the number of offices at different locations. Today, organizations are available to the public on the internet. The concept of accessing resources or seeking services is now far simpler and more rapid with the help of the internet. People now conveniently send loads of data of any format from their offices to remote data centers miles away. Although this may allow flexible communications, it also increases the risk surface area which may invite unnecessary cyber-attacks and cyber-harmful agents such as viruses, and worms into the enterprise's trusted assets to render them useless.

The concept of risk assessment involves identifying potential threats that could negatively impact an organization's ability to conduct business. These assessments help identify inherent business risks, leading to the establishment of procedures, processes, and controls to mitigate their impact on corporate operations. Organizations can employ a Risk Assessment Framework (RAF) to evaluate and communicate their risk assessment findings, encompassing threats to their information technology (IT) infrastructure. RAF assists in identifying potential hazards, identifying vulnerable business assets, and assessing the potential consequences of these risks. In larger enterprises, the oversight of the risk assessment process is typically entrusted to either the Chief Risk Officer (CRO) or the Chief Risk Manager. Threat models are tools used to identify, analyze, and mitigate risks, offering security practitioners and experts insights into what to anticipate when encountering specific threats.

The security model in the cloud is defined as a shared-responsibility model. At one end the cloud service provider (CSP) plays a role of providing security to the computing resources, along with securing the software application components shared among the tenants. It is the utmost responsibility of the CSP to secure tenant data from attackers and protect it from exposure. On the other end, the tenant is responsible for data, identity, and application software protection. Thus, these security techniques can be achieved through a systematic approach called threat modeling, defined as "The process of systematically collecting, organizing, and analyzing all relevant information that possesses a threat to an application." When applied to software, it streamlines decision-making on application security threats. Typical threat modeling efforts yield a model and a prioritized list of security changes to an application's concept, specifications, design, or implementation. All the components such as software, networks, distributed systems, Internet of Things (IoT) devices, and business processes benefit from threat modeling. As per OWASP, a threat model serves as a structured depiction of all the factors influencing the security of an application. Essentially, it provides a security-oriented viewpoint

on the application and its environment. A typical threat model incorporates the following components.

- 1) Description of the subject to be modeled
- 2) Assumptions that can be tested or questioned as the threat situation evolves
- 3) Potential threats to the system
- 4) Actions that can be taken to mitigate each threat
- 5) An approach to confirm the model's predictions and threats, as well as the effectiveness of the actions performed.

It is best to consistently use threat modeling throughout a software development project. The procedure was the same at different abstraction levels. However, information becomes increasingly granular as the life cycle progresses. A high-level threat model should ideally be created early in the design and planning phase and then refined throughout the lifecycle. New attack vectors are produced and exposed when additional features are added to the system. These dangers should be examined, diagnosed, and addressed as part of the continuing threat modeling process. Some of the advantages of threat modeling can be briefly described as follows.

Improving collaboration: First and foremost, threat modeling aids in obtaining all departments within an organization on the same page. Threat modeling ensures alignment across the IT team, executives, and key stakeholders by establishing a common foundation and shared assumptions. This is achieved through the identification of IT resources and the corresponding challenges they pose.

Reducing the attack surface: Hidden doors and other vulnerabilities in the IT environment can be identified using threat modeling and patched swiftly and efficiently. Furthermore, threat modeling assists in reducing IT complexity by identifying unnecessary endpoints, software, or resources that may be removed.

Prioritizing cybersecurity needs: Threat modeling assists businesses in determining the risks that demand the most attention and resources, in terms of time and money. For instance, when faced with multiple vulnerabilities in an IT environment, which ones should be prioritized for remediation?

Strengthening compliance: Threat modeling assists companies in adhering to data privacy and security regulations by facilitating their comprehension of potential risks to sensitive data. An illustration of this is the General Data Protection Regulation (GDPR) mandated by the European Union, which necessitates organizations to conduct a Data Protection Impact Assessment (DPIA) before initiating new projects involving the processing of personal data.

Kivioja and Sini [25] conducted an immersive analysis to identify various protection techniques applied to the assets of the organization. They examined several Risk/Threat Assessment techniques including OCTAVE, FAIR, STRIDE, and CORAS. These techniques were discussed based on their use case and application, thereby providing a comprehensive overview of threat assessment strategies. DevSecOps was explored as an approach to integrate security into the

development process. In line with the above researcher, the following subsections focuses on the threat models and architectures essential to establish a DevSecOps pipeline with automated security scanning.

A. STRIDE

One of the most prominent threat modeling architectures, STRIDE, was developed by Microsoft engineers to aid the identification of risks in a system. It is utilized in conjunction with a target system model. This makes it useful for assessing individual systems.

TABLE 1. Threats modeled by stride and their consequences.

Threat	Consequence
Spoofing Identity	Involves attackers attempting to mimic individuals or systems in order to obtain unauthorized access.
Tampering with Data	Attackers seeking to change data in transit or at rest are involved in this threat.
Repudiation	An attacker's attempt to disavow their actions or transactions within the system.
Information Disclosure	This threat entails the uninvited disclosure of sensitive information
Denial of Service	DoS attacks are designed to disrupt the availability of a service.
Elevation of Privilege	Attackers may attempt to obtain elevated privileges in order to gain access to resources that are above their standard permissions

STRIDE is an acronym for the several categories of threats that it addresses. STRIDE stands for Spoofing Tampering Repudiation of Information Disclosure Denial of Service (DoS) and Elevation of Privilege. These STRIDE threat categories are incorporated into DevSecOps' security assessments and risk management practices. This enables developers, security teams, and operations teams to collaborate in order to identify potential vulnerabilities and threats early in the development lifecycle and effectively mitigate them. The table 1 describes the threats and the prominent mitigation techniques employed using STRIDE.

B. PASTA

We now move to another useful threat model PASTA, which is designed to correlate business objectives with technical requirements. This represents the Process of Attack Simulation and Threat Analysis. PASTA's steps guide teams to dynamically identify, count, and prioritize threats. PASTA is a seven-step approach developed with an attacker-centric mindset. They can be applied to DevSecOps in the following manner.

i) *Define Business Objectives:* In the initial phase of PASTA threat modeling, the starting point involves defining the objectives. This term encompasses a broad spectrum, including business objectives related to the subject within the scope of threat modeling, the objectives of the threat modeling session, security objectives, considerations for security governance, and compliance objectives.

ii) *Define the technical scope of assets and components:* The second stage of PASTA threat modeling defines the technical scope and attack surface. This information is crucial for determining 'what we are safeguarding' from a technical perspective. This focuses on technology rather than people and procedures. Relationships with other systems, dependencies, imports, etc. are all part of the technical scope and attack surface.

iii) *Application decomposition and identification of application controls:* The third level of PASTA threat modeling is application decomposition. Decomposing entails delving deeply into the internals of an application. The outcome of this step will provide a great deal of information on how the application under consideration operates, as well as how core security concepts are applied. Another key outcome is gaining the understanding of trust, identifying the trust's existence, where it is established and determining the internal or external trust boundaries. At this level of PASTA threat modeling, data flow diagrams could be developed, similar to STRIDE. It is feasible to understand trust flows using Data Flow Diagrams. This can be used to analyze the dangers in later steps.

iv) *Threat analysis based on threat intelligence:* The fourth level of PASTA threat modeling is threat analysis. We identified the application environment and details in the previous steps. It is now time to detect and assess the potential dangers. The fourth most essential output is the awareness of threats and how they relate to the attack surface. Threats are defined based on general security knowledge, research on threat intelligence reports, known attack pathways, and consulting threat libraries. PASTA threat modeling is distinguished by the inclusion of threats based on relevancy (i.e., evidence that leads to hazardous scenarios are genuinely exploited in the real world). Furthermore, risky information originates from multiple sources rather than from a single source.

v) *Vulnerability detection:* The fifth level of PASTA threat modeling is vulnerability analysis. The primary purpose of this stage is to associate vulnerabilities with assets, resulting in a thorough awareness of the potential risks associated with threats. Vulnerabilities might originate in the code base or design through static analysis, manual penetration testing, vulnerability management, and threat intelligence.

vi) *Attack enumeration and modeling:* The sixth stage of PASTA threat modeling is the attack analysis. At this level, the identified threats and vulnerabilities are linked and proof of feasibility is provided. This improves the understanding of likelihood, which is essential for recognizing and evaluating risk.

vii) *Risk analysis and development of countermeasures:* The seventh step of PASTA threat modeling involves risk and impact analysis. PASTA is an end-to-end threat modeling technique, which means that risk reduction is an integral component of the process rather than simply recognizing risks and failing to mitigate them. Consequently, the PASTA involves specifying countermeasures to threats. The following benefits

could be observed by adapting PASTA threat modeling in the DevOps pipeline.

1) *Proactive thinking about Security*: DevOps teams are accountable for the security of their applications or IT systems as part of DevSecOps and Shift-Left security. PASTA threat modeling is a useful practice for thinking about security in advance, even if the team lacks security expertise or experience. Thus, implementing PASTA will assist a team in thinking about security proactively rather than passively in response to external triggers such as an audit, a security incident, or the need to comply with an IT control framework.

2) *Work as a team on the topic of Security*: Many individuals, including many members of DevOps teams, find security difficult. Some team members may be highly technical and possess extensive knowledge of security. However, many team members may not do so. PASTA meetings may help teams address security and promote knowledge among all team members.

3) *Determine threats early on (starting in the design phase)*: Considering potential pitfalls and anticipating them often happens late in the development lifecycle or after an application has been deployed to production. This may be too late, as suitable countermeasures to minimize the threat may be absent. PASTA threat modeling can assist in identifying possible vulnerabilities early, even during the design process. This will assist the DevOps team in considering hazards throughout the development lifecycle.

4) *Development of security requirements*: Once risks have been identified, security requirements can be developed to establish mitigations or countermeasures. The earlier this occurs, the better because security needs may be integrated earlier in the development process.

C. OCTAVE

Many organizations have established comprehensive information security control frameworks aimed at enhancing risk management and mitigation strategies, enabling them to capitalize on opportunities and achieve strategic objectives. In response to the information security compliance challenges encountered by numerous organizations, the Software Engineering Institute (SEI) introduced the Operationally Critical Threat, Asset, and Vulnerability Evaluation (OCTAVE) methodology. OCTAVE techniques were first created to solve information security challenges experienced by the United States Department of Defense (DoD), but OCTAVE have become available to the general public after proving to be successful. The primary function of OCTAVE is to assist organizations in ensuring that their information security measures are in line with their goals and objectives. The three main objectives of OCTAVE are information security evaluation, software risk management, and surveying the current state of the practice in information security risk management. OCTAVE explores technological and organizational concerns to build a complete picture of an enterprise's information security requirements. The following phases

(Figure 5) can help implement OCTAVE threat modeling for the DevOps pipeline:

Phase 1 Build Enterprise-wide Security Requirements: This phase involves analyzing enterprise-level assets and determining the assets that are important to the organization.

Phase 2 Identify Infrastructure Vulnerabilities: The second phase describes the infrastructural risk assessment and generates policies and practices to address vulnerabilities. In addition, the structure and configuration of the organizational information infrastructure is identified to examine dataflows and access paths.

Phase 3 Determine Security Risk Management Strategy: The last phase involves prioritizing the risks that must be assessed and converted into a continual security management strategy. In addition, a multidimensional risk analysis was conducted to determine the vulnerability points in potential intrusion scenarios. Although OCTAVE deals with general protection mechanisms against potential threats to the organization, a newer modified version, OCTAVE allegro can be adapted for cloud applications. The main threats identified by OCTAVE Allegro include human attackers using physical means, technical problems, and others. In the OCTAVE/Allegro threat modeling paradigm, actors, assets, access or means, motives, and consequences are among the key elements of a threat [26]. Therefore, it can be implemented in the DevOps pipeline.

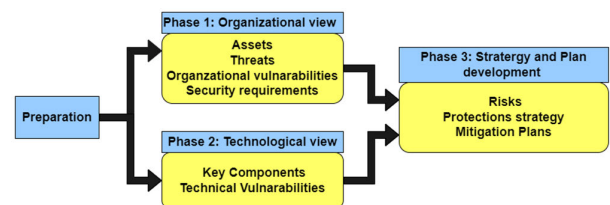


FIGURE 5. Various phases of risk assessment in OCTAVE.

D. TRIKE

Threat and Risk Intelligence Knowledge Elicitation is an open-source threat modeling paradigm that focuses on the security auditing process from the perspective of defense and risk management and considers implementation, risk models, and threats. The primary purpose of TRIKE is to ensure that stakeholders accept the risk entailment and its effects. Encouraging stakeholders to comprehend and minimize the risks of their actions within their respective domains, may pose problems to themselves and other stakeholders [27].

Similar to other techniques, TRIKE identifies and comprehends the system's actors, assets, intended actions, and rules; analysts must create a requirement model. This process leads to the creation of an actor-asset-action matrix, where the rows correspond to actors and the columns to assets. TRIKE employs a five-point scale for each activity, based on its probability, to evaluate the danger of attacks that could have an impact on assets through CRUD operations. The design and functionality of the Trike scale system's may not

be aligned with the current best practices in security measures and evaluation. Therefore, they have not been widely employed in recent security applications [28].

E. VAST

A distinctive and innovative method of cybersecurity known as “vast threat modeling” was created to address the complexity of large-scale enterprise systems. The only paradigm that allows scalability across a complete organization is VAST modeling, which stands for Visual, Agile, Simple Threat modeling. The complete Software Development Life Cycle (SDLC) is covered by Vast Threat Modeling, which relies on the three fundamental pillars of automation, integration, and collaboration. The three main components of the VAST are described below.

Visual: The visual representation of the system architecture, data flows, and interactions is emphasized in the VAST modeling. This enables stakeholders to obtain a thorough understanding of the components of the system and their interdependence. Visual models, such as diagrams or flowcharts, aid in the identification of potential security and trust boundaries within the system.

Agile: VAST approach fosters an agile attitude by fostering iterative and collaborative threat modeling. Throughout the development process, threat models are constantly refined and updated. Agile approaches, such as Scrum or Kanban, can be used to help with threat modeling and ensure its incorporation into the development workflow.

Simple: The VAST approach emphasizes simplicity in making threat modeling accessible to a broader variety of stakeholders, such as developers, architects, and business analysts. In favor of a more simplified approach, it avoids unnecessarily complex and time-consuming procedures. The emphasis is on swift and effective identification of the most critical threats and vulnerabilities.

The Agile approach of VAST can be easily adapted into an automated DevOps pipeline, through which organizations can proactively address security concerns throughout the software development lifecycle. Acknowledging operational differences and challenges between development and infrastructure teams, VAST mandates the creation of two distinct models: application and operational threat models. Application threat models utilize process-flow diagrams to illustrate the architectural perspective, whereas operational threat models, constructed from the attacker’s viewpoint, rely on Data Flow Diagrams (DFDs). This methodology facilitates the seamless integration of VAST into the development and DevOps lifecycles of an organization [29].

V. THREATS TO CLOUD SERVICE MODELS AND THEIR MITIGATION STRATEGIES

Cloud-based applications are frequently hosted in Infrastructure as Service (IaaS), Platform as Service (PaaS), and Software as Service (SaaS), making it difficult for testers to gain access to all regions of applications that require testing. Furthermore, these environments may have varying

degrees of control over how data is stored, thereby rendering it more difficult to test different attack scenarios, such as SQL injection attacks or cross-site scripting. These test initiatives risk that customer data will be compromised during tests performed by third parties, such as penetration testers, causing service interruption [30]. Therefore, it is important for both service providers and tenants to securely build their applications using security best practices. Based on the threat models elaborated in the previous section, a detailed explanation of the different approaches that can be used in the DevOps methodology is discussed in this paper. Each service model, that is, IaaS, PaaS, and SaaS has its own approach and will be discussed individually to emphasize the different attack vectors that are possible along with a few mitigation techniques. A clear representation of the same is presented in Figure 8.

A. IAAS

IaaS is a cloud computing paradigm that allows users to obtain computational resources such as servers, storage, networking, and virtualization on demand. IaaS is attractive because acquiring computing resources to store data or run complex applications requires time and capital. Moreover, it could take months for the equipment to arrive. Accommodation along with power and cooling should then be made for the equipment. IT professionals should be hired for maintenance and failure management. IaaS also provides a higher degree of management than the other service models. As IaaS provides maximum control over the virtual infrastructure to the user it is prone to various threats. These threats are identified and modeled as shown in Table 2.

1) DENIAL OF SERVICE (DOS)

DoS and Distributed-DoS attacks pose substantial dangers in a DevSecOps environment, affecting system and application availability, integrity, and performance. These malicious attacks can interrupt the normal functioning of an organization’s infrastructure, resulting in serious repercussions and financial losses. One of the most serious threats posed by DoS and DDoS attacks in DevSecOps environments is the interruption of vital services and operations. Attackers can exhaust a system’s capability to serve legitimate requests by overpowering network resources, resulting in service degradation or unavailability. This interruption has a direct influence on DevSecOps capacity to deploy, monitor, and manage services, making it difficult for them to accomplish business objectives and produce safe software. Furthermore, DoS and DDoS attacks can exploit weaknesses in the application and network layers, potentially resulting in security breaches. Such attacks can function as decoys, drawing attention away from other hostile operations that target sensitive data or seek to exploit weaknesses in the DevSecOps pipeline. DoS and DDoS attacks have a greater impact in the DevSecOps context, where quick and continuous deployment is a core premise. Organizations are particularly vulnerable to attacks

that target the availability of these systems because of the frequent supply of software updates and reliance on linked services. Such attacks can stabilize the efficient distribution of new features, upgrades, or bug patches, significantly impacting the user experience and possibly damaging a company's reputation. Load balancers and scalable infrastructure can aid in traffic distribution and attack absorption. Furthermore, frequent vulnerability assessments, penetration testing, and safe coding practices can assist in identifying and addressing vulnerabilities throughout the software development lifecycle. Some tools that can be deployed in DevSecOps are Web Application Firewalls (WAF), Network Traffic Analyzers (NTA), and Application Performance Monitoring (APM), which in turn would help minimize the attack surface for potential DoS and DDoS threats. Overall, organizations must comprehend the damage that DoS and DDoS attacks may cause in a DevSecOps context to build suitable security solutions. Organizations may better safeguard their systems, maintain service availability, and assure the integrity and performance of their software delivery processes by adopting a proactive and comprehensive approach to security.

2) DATA BREACHES

Data breaches in an Infrastructure-as-a-Service (IaaS) environment may have serious consequences for DevOps. These breaches jeopardize data security, integrity, and availability, thus placing the secure development and delivery processes at risk. First, data breaches provide unauthorized access to sensitive information such as customer data, intellectual property, or secret algorithms. This undermines the confidentiality of vital assets and can result in significant financial and reputational harm. In a DevOps environment, where sensitive data protection is crucial, such breaches weaken the organization's confidence and credibility. Second, data breaches in the IaaS environment may introduce harmful codes or jeopardize the integrity of software deployments. Attackers may use vulnerabilities to insert malware, breach containers, or tamper with software components, which might result in the deployment of compromised or malicious code. Furthermore, data breaches in an IaaS environment may influence service availability and continuous delivery pipelines. DDoS attacks may be launched by attackers, overloading resources and causing service degradation or unavailability. This significantly impairs DevSecOps ability to successfully deploy, monitor, and manage services, leading to missed deadlines, reduced efficiency, and a detrimental impact on the entire development and delivery lifecycle. Organizations must prioritize effective security measures to limit the impact of data breaches in an IaaS environment on DevSecOps. Strong access restrictions, encryption techniques, and frequent security audits may aid in protecting data at rest and in transit. Continuous vulnerability assessments, penetration testing, and system log monitoring are required to quickly identify and respond to possible attacks. Furthermore, DevSecOps teams must promote a culture of security awareness and collaboration to ensure a proactive approach of security across

development and delivery processes. The use of tools such as Security Information and Event Management (SIEM) Systems, File Integrity Monitoring (FIM), and User and Entity Behavior Analytics (UEBA) would help detect data breaches to ensure the safety of critical data. In conclusion, data breaches in an IaaS context present substantial difficulty for implementing DevSecOps. Organizations may reduce the risk of breaches, protect sensitive information, and ensure the integrity and availability of services in a DevSecOps-driven ecosystem by deploying comprehensive security measures.

3) SERVER-SIDE INJECTION (SSI)

SSIs are directives found in web applications used to populate an HTML page with dynamic content. They are akin to Computer Generated Images (CGIs), with the exception that SSIs are used to perform activities in web application before or during loading of the current page.

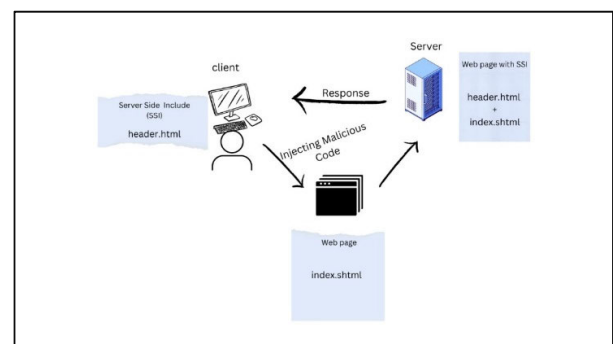


FIGURE 6. SSI attack flow.

As described in Figure 6, on the server side, potential threats involve attacks facilitating the remote execution of arbitrary code or the injection of scripts into HTML pages, thereby exploiting vulnerabilities in online applications. These attacks can occur through the manipulation of Server-Side Includes (SSI) within the application or by compelling their utilization through the user input. Notably, the success of such attacks hinges on the web server permitting SSI execution without adequate validation. If exploited, this could lead to unauthorized access, manipulation of the file system and processes, and compromise of sensitive data, including password files, with the permissions of the web server process owner. The attacker accomplishes this by inserting SSI directives into input fields, which are then transmitted to the web server. Upon parsing and executing these directives before page display, the impact of the attack becomes visible upon subsequent loading of pages into the user's browser. This SSI attack enables attackers to run arbitrary commands or queries by exploiting the server's implicit trust. SQL, OS command, and LDAP injections are some examples of common server-side injections. Attackers may be able to get around authentication, access restricted information, or even take over the entire system by exploiting this vulnerability. To mitigate potential risks and safeguard sensitive data from unauthorized access, it is essential to

TABLE 2. Threats and their possible mitigation techniques in IaaS model.

Threat/Attack	Target Platform	Consequence	Mitigation methods	Mitigation tools
Denial of Service [31]	Network Infrastructure, Servers, IoT devices	Failure to serve legitimate requests due to overwhelming traffic from bots	Methods like security architecture scanning, machine learning, and ACLs for traffic littering can be used.	Web Application Firewalls (WAF), Network Traffic Analyzers (NTA), and Application Performance Monitoring (APM)
Data Breaches [32]	Amazon EC2 based applications	Could lead to Heavy financial loss for companies along with leak of highly confidential data	Access rights, Individual behavior analysis, and Deep learning methods were used to detect.	Security Information and Event Management (SIEM) Systems, File Integrity Monitoring (FIM), and User and Entity Behavior Analytics (UEBA)
Server-Side Injection [33]	Irrespective of the Target Platform	Leads to potential access to the organization's assets or data by leveraging faulty semantic SQL commands	Input validation, strong regular expression matching along with DLPD tools help avert SSI attacks.	SAST, DAST and RASP
Hyper jacking [34]	Any Vulnerable Hypervisor	Provides a weak point in the infrastructure allowing attacks such as DoS, VM Escapes and others targeting cloud environments.	Disabling IP broadcasts, and firewalls, deploying intrusion detection systems, and applying security patches are some ways to mitigate.	Identity and Access Management (IAM) Solutions, Cloud Security Posture Management (CSPM), and Privileged Access Management (PAM)

implement appropriate input validation and sanitization techniques, use parameterized queries or prepared statements, and regularly update and patch server-side applications. Furthermore, the utilization of tools that help in discovering SSI attacks, such as SAST, DAST, and Runtime Application Self-Protection (RASP) would significantly improve the overall security of the application.

4) HYPER JACKING

Hyper-Jacking attacks on Infrastructure-as-a-Service (IaaS) in a DevSecOps environment have serious consequences, positioning the security and integrity of software development and delivery processes at risk. Unauthorized access to and control of an organization's IaaS infrastructure is referred to as hyper jacking. Such attacks can result in serious ramifications. First, hyper jacking compromises the security and privacy of sensitive data stored in an IaaS environment. Attackers may acquire unauthorized access to consumer data, confidential information, or intellectual property, thereby causing severe financial and reputational harm.

Secondly, compromised IaaS resources may be used to conduct additional attacks or to spread malware across the DevSecOps pipeline. To introduce malicious malware, compromise software components, or control the development and deployment process, attackers may use compromised infrastructure. This places the integrity of the software being built and supplied at risk, making it harder to sustain the ecosystem's trustworthiness. Furthermore, hyper jacking might influence service availability and the continuous supply pipeline. Attackers may take advantage of

the hijacked computers. Attackers may use stolen assets to perform distributed denial-of-service (DDoS) attacks, overloading systems and causing service degradation or total outages. This has a direct influence on DevSecOps' capacity to provide and maintain services, resulting in missed deadlines, lower productivity, and a detrimental impact on the whole software development lifecycle. Furthermore, in the DevSecOps context, hyper jacking inhibits collaboration and confidence between development, security, and operations teams. Breach incidents require reactive incident response and mitigation activities, devoting significant time and resources away from proactive security measures. This strain of team chemistry impedes efficient communication, and makes it difficult to incorporate strong security practices throughout the development process. Organizations must prioritize robust security measures to prevent the impact of hyper jacking attacks on DevSecOps in an IaaS environment. Multi-factor authentication, rigorous access limits, and frequent security audits all assist in safeguarding against unauthorized access. Data integrity and confidentiality are ensured by employing advanced network monitoring, intrusion-detection systems, and encryption procedures. Popular tools that help detect hyper jacking attacks early include Identity and Access Management (IAM) Solutions, Cloud Security Posture Management (CSPM), and Privileged Access Management (PAM). Continuous vulnerability assessments, penetration testing, and timely patch management are required to quickly detect and resolve vulnerabilities. To summarize, hyper jacking attacks on IaaS in the DevSecOps context pose substantial threats to software development, delivery, security,

and integrity. Organizations may limit risks, secure sensitive data, and ensure the integrity and availability of services in a DevSecOps-driven ecosystem by deploying rigorous safety protocols.

B. PAAS

PaaS represents a Cloud computing paradigm that furnishes a flexible and scalable cloud platform for the creation, deployment, execution, and management of applications. This approach streamlines the application development process for developers by alleviating concerns related to the operating system update, managing development tools, and maintaining hardware. In lieu of these responsibilities, third-party service provider delivers the entire PaaS environment or platform. To explore the major threats in a PaaS environment, as outlined in Table 3, the following sections provide detailed insights.

1) VULNERABLE APPLICATION CODE

Vulnerable application code in a platform-as-a-service (PaaS) environment might have a negative impact on DevSecOps principles, increasing the security and dependability of software development and delivery processes. Several hazards occur when the application code has vulnerabilities in a PaaS environment. First, an insecure code opens the door for attackers to exploit and infiltrate the program. This can result in unauthorized access, data breaches, or malicious activities within the PaaS platform. Vulnerable code jeopardizes the integrity and confidentiality of important assets and sensitive information in a DevSecOps environment where secure development practices are necessary.

Secondly, flaws in the application code may lead to the exploitation of platform flaws or misconfigurations. Insecure code can be used by attackers to circumvent security restrictions, elevate privileges, or obtain unauthorized access to platform resources. Furthermore, in a PaaS environment, insecure application codes impede the agility and efficiency of DevSecOps pipelines. Security problems can cause delays and interruptions in the software development lifecycle, necessitating more time and effort to identify and fix vulnerabilities. This can result

in missed deadlines, decreased productivity, and general slowdown in the delivery of safe software. Organizations should prioritize rigorous security practices to limit the impact of risky application codes in a PaaS environment on DevSecOps. Introducing safe coding practices such as regular code reviews, static and dynamic analysis, and secure coding principles aids in the early detection and mitigation of vulnerabilities. This decreases the chance of an insecure code being deployed in a PaaS environment.

Thirdly, employing security testing methodologies, such as penetration testing and vulnerability scanning, allows for the detection of flaws and vulnerabilities in the application code. Regular testing promotes continuous progress and the early detection of possible problems. This includes features such as access restrictions, encryption, and logging. All these

features can be used to successfully protect deployed applications. In conclusion, insecure application codes in a PaaS context pose considerable threats to DevOps. Organizations may minimize vulnerabilities and improve the security and dependability of software development and delivery in a PaaS-based DevOps environment using safe coding practices, conducting frequent security testing, and cultivating a culture of security awareness. Some tools that help detect vulnerable application code include Interactive Application Security Testing (IAST), Dependency Scanning Tools, and Software Composition Analysis (SCA).

2) BUILD MACHINE ATTACK

Build machine attacks pose a significant threat to the integrity and security of containerized deployments. When attackers gain the capability to manipulate or exert influence over the building process of container images, they can easily introduce malicious code into the production environment. Such insidious code execution can result in severe consequences including unauthorized access, data breaches, and service disruptions. Moreover, compromising the built environment can serve as a stepping stone for attackers to infiltrate and compromise the manufacturing environment, thereby causing widespread damage. Figure 7 highlights the attack zone at the CI server during the image-building process. If malware exists within an image at the time of building, it causes immense damage to the container and application. Assuming that other containers are built upon the same container, they can penetrate those containers and increase the spread of the attack. To mitigate this risk, it is crucial to implement robust security measures, including strict access controls, continuous monitoring, and secure build processes, to safeguard the integrity of the built environment and prevent the inclusion of malicious code in container images.

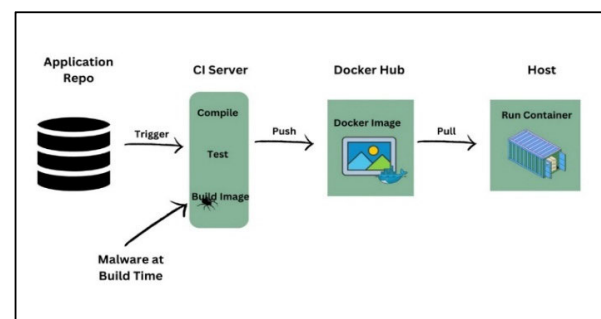


FIGURE 7. Build machine attack.

3) VULNERABLE HOSTS

Vulnerable hosts in a PaaS environment can negatively impact the security and resilience of DevSecOps practices, sacrificing the integrity and availability of software development and delivery processes. Several dangers and issues exist when hosts in a PaaS environment are susceptible. For instance, attackers can exploit insecure hosts to obtain unauthorized access to the underlying infrastructure. This can lead to

TABLE 3. Threats and their possible mitigation techniques in PaaS model.

Threat/Attack	Target Platform	Consequence	Mitigation methods	Mitigation tools
Vulnerable Application Code [35]	Physical server, a virtual machine, a containerized environment, or a cloud-based platform	The stored data is overwritten with encrypted versions using platform API, which in turn is held for ransom	Static code analysis along with dynamic code analysis are a few inspections to discover stale interfaces of applications	Interactive Application Security Testing (IAST), Dependency Scanning Tools, and Software Composition Analysis (SCA).
Build machine Attacks [36]	Mobile applications, web applications	unexpected code inside data can be automatically and mistakenly triggered	Using safe APIs, different ways to display data and excellent coding practices can reduce the chances	JFrog Xray, TwistLock, Docker Security Scanning.
Vulnerable Hosts [37]	Any machine with weak security patches	leads to the possibility for attackers to get access to an organization's resources or data.	Optimal Scanning Strategy to detect vulnerable hosts with random scanning strategy	Patch Management Solutions, Secure Shell (SSH) Auditing and Hardening
Exposed Secrets [38]	Cloud Platforms, Web Applications, Mobile applications, containerized environments	Alters timing of the process and exposes credentials of web applications to illegitimate users	Modification in OS level kernel variables for granular flow along with proof testing API calls	Secret Management Tools (SMT), Credential Rotation

unauthorized data access, privilege escalation, or jeopardization of crucial resources. and imperils the confidentiality, integrity, and availability of deployed applications and services in a DevSecOps environment, where secure infrastructure is crucial. Second, host vulnerabilities may compromise the entire PaaS system. Attackers may use vulnerabilities to conduct additional attacks, spread malware, or impair platform operation. This jeopardizes the stability and security of the DevSecOps pipeline, hindering application deployment, monitoring, and maintenance. In addition, insecure hosts in a PaaS environment make it difficult to maintain secure and compliant architecture. Host configuration flaws, out-of-date software versions, and missing patches can result in security breaches and non-compliance with industry standards and regulations. This endangers sensitive data and exposes organizations to legal and financial implications. Organizations should prioritize robust security practices to limit the impact of insecure hosts in a PaaS environment on DevSecOps. For the safe and resilient DevSecOps practice, proactive vulnerability management, patch management, and configuration hardening are essential. Access restrictions, network segmentation, logging, monitoring, and intrusion detection systems aid in the discovery and prevention of suspicious behaviors. Organizations may improve the security and reliability of software development and delivery processes in the PaaS-based DevSecOps ecosystem by increasing security awareness and addressing these threats. Further Employment of Patch Management Solutions, Secure Shell (SSH) auditing and hardening, and endpoint protection should mitigate most of the risks involved.

4) EXPOSED SECRETS

Exposed secrets in a platform-as-a-service (PaaS) environment pose considerable threats to the sensitive information

security and confidentiality of DevSecOps. When secrets such as API keys, passwords, or cryptographic keys, are exposed in a PaaS context, several issues and vulnerabilities develop. Firstly, leaked secrets may result in unauthorized access and significant data breaches. Attackers with access to disclosed secrets can use them to obtain unauthorized access to critical components, compromise sensitive data, or conduct harmful actions on the PaaS platform. Exposed secrets jeopardize the confidentiality and integrity of important assets in a DevSecOps environment where data protection is crucial. Secondly, disclosed secrets increase the possibility of insider threats. Internal workers or malevolent insiders can utilize secrets available to unauthorized persons or disclosed to the public to obtain unauthorized rights, alter systems, or exfiltrate-sensitive data. Exposed secrets may also result in account takeover and unauthorized access to cloud resources. Attackers can impersonate authorized users, acquire control of PaaS resources, and even interrupt software development and delivery processes using exposed credentials or keys. This has an impact on the availability and dependability of services and undermines the key DevSecOps concepts. Organizations must use secure secret management practices such as encrypted key vaults and secret management services, to reduce vulnerabilities in a PaaS environment for DevSecOps. Strict access restrictions and robust authentication systems guarantee that only authorized individuals can access and manage secrets. To safeguard sensitive information, avoid hardcoding secrets, use environmental variables, and use encryption or hashing methods such as Secret Management Tools (SMT), credential rotation, Employee Training and Security Awareness programs. Organizations may increase their security posture inside the PaaS-based DevSecOps ecosystem by applying these strategies.

TABLE 4. Threats and their possible mitigation techniques in SaaS model.

Threat/Attack	Target Platform	Consequence	Mitigation methods	Mitigation tools
Ransomware [41]	AWS S3-based applications	The stored data is overwritten with encrypted versions using platform API, which in turn is held for ransom	Usage of repeated decryption techniques like AES to mitigate the attack	AWS Blueprint for Ransomware Defense.
Zero-day attack [42]	Google Cloud platform.	Could lead to the Infiltration of malware, spyware, or unwanted access to user information	Using decoy systems like honeypots, constant monitoring, and network behavioral analysis	AWS custom patch baseline for Critical and Important Security patches
SQL Injection attack [43]	Any application which uses SQL server	Leads to potential access to the organization's assets or data by the attackers.	Adapting feature engineering and machine learning techniques like Random Forest and AdaBoost classifier to detect malicious codes	OWASP ZAP and Burp Suite
Authentication issues [44]	Any application with keys and password protection	Provides a weak point in the application for the attackers to target, using tools like a keylogger.	Usage of multi-factor authentication methods, and Access management controls to prevent unauthorized access	Okta, Azure Active Directory, Auth0, AWS IAM

C. SAAS

SaaS stands for software-as-a-service. The software provider owns and maintains the program under the SaaS model, and the consumer does not need to install any software on the computer. In this strategy, software is typically licensed on a subscription basis. To compete, businesses use SaaS to generate money, pivot their products to consumer demand, optimize current apps, innovate, and swiftly go to the market. In short, SaaS allows full control with the cloud providers and only delivers the final application to the user, for which user has control over the application but not the underlying technology. Although most of the infrastructure is maintained at the host's end, there may be some threats that need to be addressed in the SaaS service model, as listed in Table 4. Further elaboration of these threat and attack models is presented below.

1) RANSOMWARE ATTACK

A Ransomware attack is most commonly directed at cloud-based email services such as Office 365. The attacker uses phishing tactics to acquire access to email accounts, encrypt correspondence, and demand a ransom. Attackers will also try to mimic the account owner to fool the victim's contacts to install and spread ransomware application. A recent attack observed by cybersecurity firm Obsidian targeting SharePoint Online (Microsoft 365) demonstrated the vulnerability of SaaS platforms to such attacks. Although these attacks can be mitigated by carefully monitoring all the vulnerabilities in the pipeline or software artifacts, they still pose a major challenge to individuals and organizations. One of the prominent tools and best practices that can be utilized is the AWS Blueprint for Ransomware defense [39].

2) ZERO-DAY ATTACK

A zero-day vulnerability is a security flaw in software that developers have not yet identified or patched. Cybercriminals can exploit these vulnerabilities through cyber-attacks termed zero-day attacks, resulting in data breaches and data loss across affected organizations. The term "zero-day" alludes to the fact that the nature of vulnerability is unknown to individuals who are not attackers, notably developers. Once a vulnerability is discovered, the developer must scramble to safeguard users. The software business must produce a patch for the vendor to fix its vulnerability. Zero-day attacks are a serious concern because they are novel (anomalous) threats that exploit previously discovered system vulnerabilities. One of the best practices for safeguarding applications from these attacks is same-day security patching. Tools such as the AWS custom patch baseline for Critical and Important Security patches can be configured to mitigate all immediate threats.

3) SQL INJECTION ATTACK

SQL injection is a type of security vulnerability that allows an attacker to modify the SQL query input of an application to execute malicious SQL commands in the database. SQL injection attacks in cloud contexts target web applications hosted on cloud platforms or software that connects with cloud-based databases. These attacks must be properly addressed with multiple security monitoring techniques such as isolation of applications, and channel protection. Monolithic and distributed implementation and constant monitoring for anomaly detection. The use of DAST tools such as OWASP ZAP or Burp Suite, can help identify vulnerabilities that could lead to SQL injection. Further usage

of security libraries such as OWASP, ESAPI (Enterprise Security API) could help in validating user input.

4) AUTHENTICATION ISSUES

Authentication and access management are the major security layers in many organizations. Most user-centric services still use a simple authentication involving usernames and passwords. Some well-developed organizations have deployed various additional authentication methods such as virtual keyboards and shared secret questions, to ensure common phishing attempts are more challenging. These issues create weak points that adversaries can exploit using attacks like man-in-the-middle, keylogging, and phishing. Therefore, strengthening application security is important for developers; however, adapting to best-securing techniques remains a challenging task. Today there are multiple tools offered by many third-party services including Single Sign-On (SSO) Solutions such as Okta, Azure Active Directory, Auth0, Identity and Access Management (IAM) tools, which help manage user access permissions and enforce the principle of least privilege. Further implementation of logging and monitoring using tools such as ELK Stack (Elasticsearch, Logstash, and Kibana) to track and investigate authentication-related events which in turn minimize authentication issues. Penetration testing is one of the most prominent security techniques used in SaaS applications. However, this brings about some of the major problems. The first challenge is to understand how SaaS applications differ from traditional on-premise systems. As these cloud-native applications offer multitenancy and shared services, the complexity of the systems increases. Therefore, organizations lack the knowledge and infrastructure to evaluate backend security systems [40].

VI. DEVSECOPS IN MULTI-CLOUD ARCHITECTURE

The STRIDE, PASTA, OCTAVE, VAST, and TRIKE frameworks are collectively used as threat modeling techniques in software development. The proposed system denotes these frameworks by set F , where: $F = \{\text{STRIDE, PASTA, OCTAVE, VAST, TRIKE}\}$. In a Multi-Cloud Architecture, the set of cloud providers is denoted by C , where C includes multiple cloud providers: $C = \{C_1, C_2, \dots, C_n\}$. The proposed objective is to secure Multi-Cloud Environments denoted as Secure(MCA) by mitigating any threats or challenges that arise in these Environments.

A. MULTI-CLOUD SECURITY CHALLENGES: ADOPTING TRADITIONAL THREAT MODELS

Threat modeling frameworks (F) have been extensively applied within the realm of software development to enhance security protocols.

The primary objective underlying the utilization of these frameworks is to fortify the security posture inherent to a Single Cloud Architecture (SCA) environment, which is achieved through the meticulous identification of potential threats and vulnerabilities within said environment. The adoption of these methodologies by an SCA confers

numerous advantages, encompassing but not limited to enhanced risk assessment, meticulous vulnerability identification, implementation of proactive security measures, facilitation of threat prioritization, and the provision of a systematic approach to resolving security issues. Nevertheless, threat models associated with SCA have a number of limitations when applied to multi-cloud environment. The limitations are outlined as follows.

Visibility Oversight (V): With multiple cloud providers and their distinct security tools, gaining a comprehensive view of potential threats across the entire environment can be a major hurdle. Deploying a mission-critical application with components across both AWS and Azure can be advantageous by leveraging the strengths of each platform. However, achieving seamless correlation between performance metrics and security events across disparate cloud environments presents a significant challenge. While each platform offers individual monitoring tools, seamlessly correlating the performance metrics and security events across both clouds can be difficult. A data breach originating in one cloud may remain undetected until it impacts the other, creating a significant risk. Therefore, this challenge can be represented as $V = \{\text{Lack of a comprehensive view across } C_i \text{ and } C_j \mid C_i, C_j \in C\}$

Interconnected Threats (I): Multi-cloud setups are linked; thus, a vulnerability in one cloud may have a cascade effect on another. For example, a compromised database in Microsoft Azure connected to a critical application hosted on IBM Cloud can provide attackers with an entry point to infiltrate the entire system. The lack of a unified security posture across both platforms can hinder an effective incident response and containment. For threat models to evaluate the entire security posture, interdependencies must be considered. These interconnected threats can be represented as, $I = \{\text{Vulnerability in } C_i \text{ affecting } C_j \mid C_i, C_j \in C\}$

Hidden Costs and Management Complexity (H): Initial cost savings in a Multi-Cloud Architecture (MCA) may seem attractive; however, unforeseen charges stemming from divergent vendor pricing structures can escalate. A research institution using both GCP for high-performance computing and AWS for data storage might incur hidden costs owing to unused resources, complex licensing agreements, and manual effort required for cross-cloud resource management. These hidden costs can be expressed as $H = \{\text{Additional costs and management complexity across } C_i, C_j, \dots \mid C_i, C_j, \dots \in C\}$

Automation Enigma (A): In multi-cloud setups, automating threat modeling can be challenging because of the lack of standardized tools and APIs. Consider an organization that intends to automate threat modeling as part of its DevSecOps pipeline in a multi-cloud infrastructure comprising of platforms such as AWS, Azure, and Google Cloud. The lack of standardized tools across these disparate cloud providers hinders automation which can be represented as $A = \{\text{Challenges in automating threat modeling across } C_i, C_j, \dots \mid C_i, C_j, \dots \in C\}$. As each cloud platform has its own set of security rules, API architectures, and data formats, it can be difficult

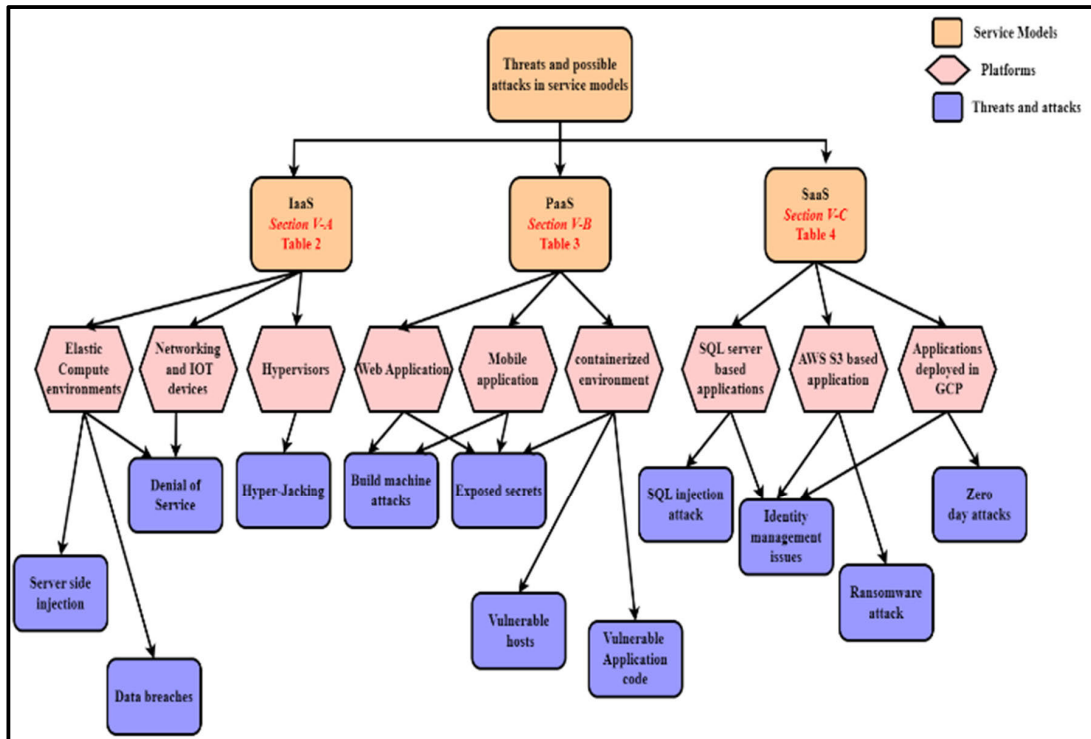


FIGURE 8. Threats and possible attacks in different service models.

to integrate threat modeling into an automated workflow. It's critical to identify the ideal set of instruments and procedures for integrating threat modeling into the DevSecOps pipeline.

B. LAYERED ARCHITECTURE FOR DEVSECOPS IN MULTI-CLOUD ENVIRONMENT

To address the multi-cloud security challenges this study proposes a layered architecture that considers various modern technologies to enable the smooth automation of a Multi-cloud Environment. This includes the following layers (Figure 9).

1) FOUNDATION LAYER (L_1)

Let the Foundational layer be denoted as $L_1 = \{\text{CSPM, CASB, IAM}\}$ with each of its components explained below.

a) *Cloud Security Posture Management (CSPM)*: CSPM serves as a centralized command center, ensuring comprehensive oversight and control of security configurations across diverse cloud providers (AWS, Azure, GCP, etc.). It continuously monitors compliance, identifies security misconfigurations, and recommends remediation strategies.

b) *Cloud Access Security Broker (CASB)*: Functioning as a stringent gatekeeper, CASB enforces security policies for cloud applications and data access across all environments. It meticulously manages user and application access, prevents unauthorized data exfiltration, and filters out malicious traffic.

c) *Identity and Access Management (IAM)*: IAM functions as the cornerstone for managing user authentication and

authorization uniformly across all cloud platforms. It precisely defines access privileges, ensuring that only authorized personnel can execute specific actions.

2) DEVSECOPS INTEGRATION LAYER (L_2)

Let the DevSecOps Integration Layer be represented as $L_2 = \{\text{SIEM, VM, TMT}\}$ where each of its components can be elaborated as:

a) *Security Information and Event Management (SIEM)*: SIEM aggregates and analyzes security data from all cloud environments, identifying potential threats and incidents in real-time. It correlates events from various sources, providing a holistic view of systems' security posture and enabling prompt incident response.

b) *Vulnerability Management (VM)*: Vulnerability management tools automate vulnerability scanning and patching across all cloud resources, proactively addressing potential security gaps before they can be exploited.

c) *Threat Modeling Tools (TMT)*: Picture a strategic war council meticulously assessing potential attack vectors. Threat modeling tools, like STRIDE and PASTA, integrate seamlessly into the development process, enabling proactive identification and mitigation of potential threats early in the software development lifecycle.

3) AUTOMATION AND ORCHESTRATION LAYER (L_3)

Let the Automation and Orchestration Layer be denoted as $L_3 = \{\text{DT, SAT, CMT}\}$, comprising the following components:

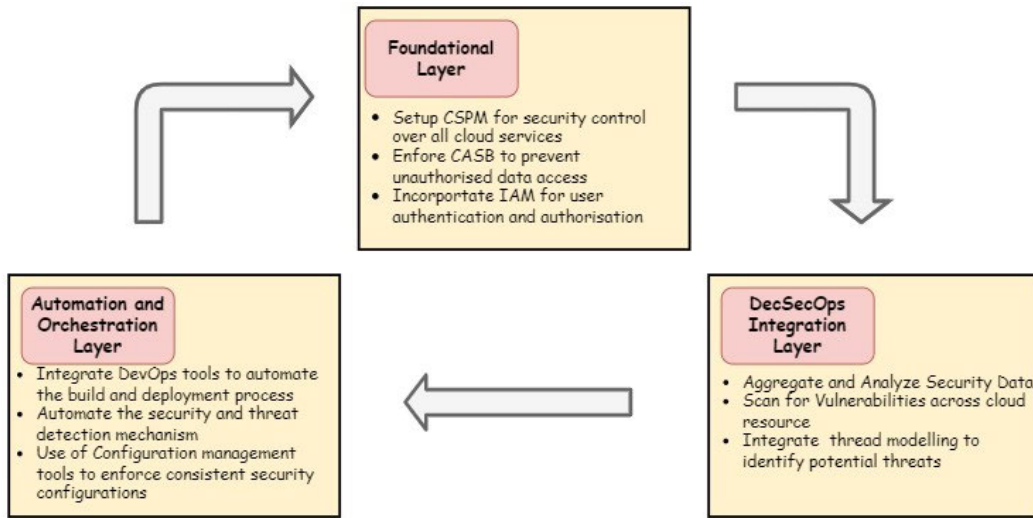


FIGURE 9. Proposed layered architecture approach for multi-cloud architecture.

a) *DevSecOps Tools (DT)*: These tools automate build, test, and deployment processes, integrating security seamlessly into the development lifecycle.

This enables rapid and secure software delivery, minimizing human error and potential vulnerabilities. Imagine DevSecOps tools as your team of automation experts, ensuring smooth and secure software deployments across your multi-cloud environment.

b) *Security Automation Tools (SAT)*: Security automation tools automate security tasks like threat detection, incident response, and remediation across all cloud environments. This frees up your security team to focus on strategic initiatives while ensuring swift and effective responses to security threats. Imagine security automation as your team of vigilant robots, tirelessly scanning for threats and automatically neutralizing them.

c) *Configuration Management Tools (CMT)*: Configuration management tools enforce consistent security configurations across all cloud resources, ensuring uniform security posture and minimizing misconfigurations. Imagine configuration management as your architect, meticulously designing and enforcing the secure blueprint for your multi-cloud environment. Therefore, the overall security posture achieved by combining the layered architecture functions across all cloud providers can be expressed as,

$$secure(MCA) = \bigcup_{C_i \in C} Layer(L_1, L_2, L_3)$$

where $Layer(L_1, L_2, L_3) = \{CSPM(C_i), CASB(C_i), IAM(C_i), SIEM(C_i), VM(C_i), TMT(F, C_i), DevOps(C_i), SAT(C_i), CMT(C_i) \mid C_i \in C\}$. The successful implementation of $secure(MCA)$ must mitigate: $\{V, I, H, A\}$. This represents the challenges of visibility oversights, interconnected threats, hidden costs and management complexity, and automation enigma that must be addressed for effective security in a multi-cloud environment.

VII. APPLICATIONS AND USE CASES

The aim of this paper was to educate the reader about the entire background necessary to ensure the delivery of secure software, from the development phase. There are many applications of DevOps, but DevSecOps are few because security is a time-consuming activity. This can be easily demonstrated by reports presented in the introduction of this paper. Not only is security a time-consuming process, but it is also an elaborate one. Only a few security personnel are capable of performing complex security tasks that encompass all aspects related to security. Nevertheless, security is now considered an add-on feature and is maintained as a last-minute activity. Use cases exist not only in the corporate world but also in the Operation Technology (OT) industry. Many manufacturers are now switching to smart appliances and machines connected to networks. This is a significant boost in terms of availability, remote access, and provision of accurate system metrics that eventually aid in repair and maintenance. A few applications and their use cases are provided below:

A. DEVOPS IN THE IT WORLD

DevOps, a successful technique, lays the groundwork for increased productivity, efficiency, and shorter cycle times by integrating development and operation teams. This finding broadens its relevance to numerous domains within the IT industry.

B. CORPORATE WORLD AND OPERATION TECHNOLOGY (OT) INDUSTRY

DevSecOps use cases expand beyond the corporate environment and into Operation Technology (OT) business.

Manufacturers in this business are increasingly embracing smart appliances and network-connected devices [45]. As described in Figure 10, the cybersecurity framework provides advantages such as increased availability,

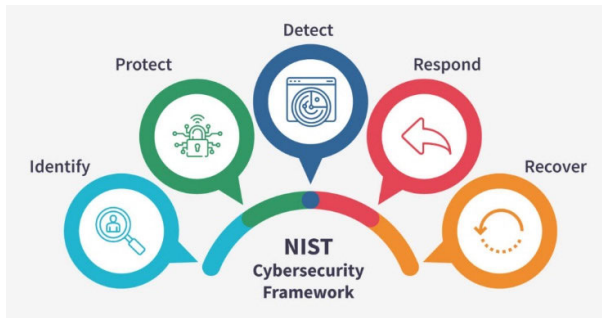


FIGURE 10. Cybersecurity in OT industry adapted from [43].

remote access, and precise system metrics, allowing for more effective repair and maintenance operations.

C. DEVSECOPS PIPELINE

The DevSecOps pipeline is a condensed set of strategies and methodologies that simplifies security practices. Organizations of any size can achieve security success in a variety of scenarios by combining a threat model analysis with a DevSecOps pipeline. Proven solutions and practices have become engrained as standard processes, propelling the field forward. An elaborate understanding of the connection between DevSecOps and threat models can potentially disrupt outdated security practices. This integration unlocks new possibilities in the cybersecurity and digital forensics domains, leading to enhanced security measures in the IT industry.

VIII. CONCLUSION & FUTURE SCOPE

This study primarily focuses on the detailed examination of threat models within DevSecOps, reinforcing the security pipeline. As this field continues to evolve, cybersecurity professionals are uncovering new opportunities for exploration and innovation. Gaining a deep understanding of DevSecOps and its role in organizational security has become a crucial step in enhancing cybersecurity awareness and defense. Given the complexities of securing diverse cloud environments, a proposed multi-layered architecture addresses these challenges by embedding DevSecOps principles throughout the development lifecycle. This research systematically maps potential threats across every layer of the infrastructure, utilizing comprehensive threat models to simulate worst-case scenarios and identify vulnerabilities. This proactive approach if implemented, enables us to fortify weaknesses before they become exploitable, building fault-tolerant code that gracefully handles unexpected situations. Unlike traditional, reactive security measures, DevSecOps empowers the system to build security directly into the proposed layered architecture, resulting in a robust and resilient multi-cloud environment.

Despite these advancements, several challenges remain, paving a way for future study. Enhancing the current threats models to work on ever changing landscape of multi-cloud becomes crucial. Adopting ZTA (Zero Trust Architecture)

can further reinforce authentication, authorization, and access control mechanisms across distributed infrastructures. Focusing on zero-day vulnerabilities, predictive threat analysis, and automated incident response can further strengthen the security posture of software development.

ACKNOWLEDGMENT

The authors sincerely appreciate the unwavering support from PES University throughout this research, including the generous support for open access publication.

REFERENCES

- [1] S. Kraemer, P. Carayon, and J. Clem, "Human and organizational factors in computer and information security: Pathways to vulnerabilities," *Comput. Secur.*, vol. 28, no. 7, pp. 509–520, Oct. 2009.
- [2] *ITRC_2023-Annual-Data-Breach-Report*. Accessed: Dec. 12, 2023. [Online]. Available: https://www.idtheftcenter.org/wpcontent/uploads/2024/01/ITRC_2023-Annual-Data-Breach-Report.pdf
- [3] *Cybersecurity Insiders*. Accessed: Jul. 4, 2023. [Online]. Available: <https://www.cybersecurity-insiders.com/wp-content/uploads/2021/06/2021-Insider-Threat-Report-Gurukul-Final-dd8f5a75.pdf>
- [4] Gartner. *Gartner Identifies the Top Cybersecurity Trends for 2023*. Accessed: Jul. 4, 2023. [Online]. Available: <https://www.gartner.com/en/newsroom/press-releases/04-12-2023-gartner-identifies-the-top-cybersecurity-trends-for-2023>
- [5] D. S. Battina, "Best practices for ensuring security in DevOps: A case study approach," *Int. J. Innov. Eng. Res. Technol.*, vol. 11, no. 11, pp. 38–45, 2017.
- [6] *The State of Devsecops Report—Contrast Security*. Accessed: Jul. 12, 2023. [Online]. Available: <https://www.contrastsecurity.com/hubfs/DocumentsPDF/The-State-of-DevSecOpsReportFinal.pdf>
- [7] Markets Markets. *DevSecOps Market Size, Share, Trends and Industry Analysis*. Accessed: Jul. 12, 2023. [Online]. Available: <https://www.marketsandmarkets.com/Market-Reports/devsecops-market-122458587.html>
- [8] X. Zhou, R. Mao, H. Zhang, Q. Dai, H. Huang, H. Shen, J. Li, and G. Rong, "Revisit security in the era of DevOps: An evidence-based inquiry into DevSecOps industry," *IET Softw.*, vol. 17, no. 4, pp. 435–454, Aug. 2023.
- [9] J. Y. Zhang and Y. Zhang, "Quantitative DevSecOps metrics for cloud-based web microservices," *IEEE Access*, vol. 12, pp. 160317–160342, 2024.
- [10] A. M. Putra and H. Kabetta, "Implementation of DevSecOps by integrating static and dynamic security testing in CI/CD pipelines," in *Proc. IEEE Int. Conf. Comput. Sci. Inf. Technol. (ICOSNIKOM)*, Oct. 2022, pp. 1–6.
- [11] A. Caniglia, V. Dentamaro, S. Galantucci, and D. Impedovo, "FOBICS: Assessing project security level through a metrics framework that evaluates DevSecOps performance," *Inf. Softw. Technol.*, vol. 178, Feb. 2025, Art. no. 107605.
- [12] X. Sun, Y. Cheng, X. Qu, and H. Li, "Design and implementation of security test pipeline based on DevSecOps," in *Proc. IEEE 4th Adv. Inf. Manage., Communicates, Electron. Autom. Control Conf. (IMCEC)*, vol. 4, Jun. 2021, pp. 532–535.
- [13] J. McZara, S. Kafle, and D. Shin, "Modeling and analysis of dependencies between microservices in DevSecOps," in *Proc. IEEE Int. Conf. Smart Cloud (SmartCloud)*, Nov. 2020, pp. 140–147.
- [14] H. Yasar, "Experiment: Sizing exposed credentials in GitHub public repositories for CI/CD," *IEEE Cybersecurity Develop. (SecDev)*, Sep. 2018, vol. 7, p. 143.
- [15] P. Bajpai and A. Lewis, "Secure development workflows in CI/CD pipelines," in *Proc. IEEE Secure Develop. Conf. (SecDev)*, Oct. 2022, pp. 65–66.
- [16] J. Mahboob and J. Coffman, "A kubernetes CI/CD pipeline with Asylo as a trusted execution environment abstraction framework," in *Proc. IEEE 11th Annu. Comput. Commun. Workshop Conf. (CCWC)*, Jan. 2021, pp. 0529–0535.
- [17] X. Ramaj, "A DevSecOps-enabled framework for risk management of critical infrastructures," in *IEEE/ACM 44th Int. Conf. Softw. Eng., Companion Proc. (ICSE-Companion)*, May 2022, pp. 242–244.

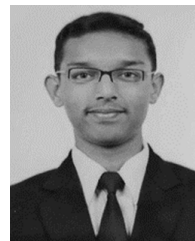
- [18] M. A. Aljohani and S. S. Alqahtani, "A unified framework for automating software security analysis in DevSecOps," in *Proc. Int. Conf. Smart Comput. Appl. (ICSCA)*, Feb. 2023, pp. 1–6.
- [19] A. Ibrahim, A. H. Yousef, and W. Medhat, "DevSecOps: A security model for infrastructure as code over the cloud," in *Proc. 2nd Int. Mobile, Intell., Ubiquitous Comput. Conf. (MIUCC)*, May 2022, pp. 284–288.
- [20] A. Sojan, R. Rajan, and P. Kuvaja, "Monitoring solution for cloud-native DevSecOps," in *Proc. IEEE 6th Int. Conf. Smart Cloud (SmartCloud)*, Nov. 2021, pp. 125–131.
- [21] J. Díaz, J. E. Pérez, M. A. Lopez-Peña, G. A. Mena, and A. Yagié, "Self-service cybersecurity monitoring as enabler for DevSecOps," *IEEE Access*, vol. 7, pp. 100283–100295, 2019.
- [22] S. Carturan and D. Goya, "Major challenges of systems-of-systems with cloud and DevOps—A financial experience report," in *Proc. IEEE/ACM 7th Int. Workshop Softw. Eng. Syst.-Syst. (SESOS) 13th Workshop Distrib. Softw. Develop., Softw. Ecosystems Syst.-Syst. (WDES)*, May 2019, pp. 10–17.
- [23] M. Cankar, N. Petrovic, J. P. Costa, A. Cernivec, J. Antic, T. Martincic, and D. Stepec, "Security in DevSecOps: Applying tools and machine learning to verification and monitoring steps," in *Proc. Companion ACM/SPEC Int. Conf. Perform. Eng.*, Apr. 2023, pp. 201–205.
- [24] N. S. Pecka, "Making secure software insecure without changing its code: The possibilities and impacts of attacks on the DevOps pipeline," Ph.D. dissertation, Iowa State Univ., Ames, IA, USA, 2022.
- [25] S. Kivioja, "Information security threat and risk assessments in DevOps," *Inf. Technol.*, 2022.
- [26] D. J. Bodeau, C. D. McCollum, and D. B. Fox, "Cyber threat modeling: Survey, assessment, and representative framework," *Mitre Corp., Mclean*, 2018.
- [27] L. O. Nweke and S. Wolthusen., "A review of asset-centric threat modelling approaches," *Int. J. Adv. Comput. Sci. Appl.*, vol. 11, no. 2, pp. 1–6, 2020.
- [28] N. Shevchenko, T. A. Chick, P. O'Riordan, T. P. Scanlon, and C. Woody, "Threat modeling: A summary of available methods," *Softw. Eng. Inst., Pittsburgh, PA, USA, Tech. Rep.*, 2018.
- [29] S. G. Abbas, S. Zahid, F. Hussain, G. A. Shah, and M. Husnain, "A threat modelling approach to analyze and mitigate botnet attacks in smart home use case," in *Proc. IEEE 14th Int. Conf. Big Data Sci. Eng. (BigDataSE)*, Dec. 2020, pp. 122–129.
- [30] N. Shevchenko. (Dec. 3, 2018). *Threat Modeling: 12 Available Methods*. *Softw. Eng. Institute's Insights (Blog)*. [Online]. Available: <https://insights.sei.cmu.edu/blog/threat-modeling-12-available-methods/>
- [31] A. Bhardwaj, V. Mangat, R. Vig, S. Halder, and M. Conti, "Distributed denial of service attacks in cloud: State-of-the-art of scientific and commercial solutions," *Comput. Sci. Rev.*, vol. 39, Feb. 2021, Art. no. 100332.
- [32] L. Cheng, F. Liu, and D. Yao, "Enterprise data breach: Causes, challenges, prevention, and future directions," *Wiley Interdiscipl. Rev., Data Mining Knowl. Discovery*, vol. 7, no. 5, 2017, Art. no. e1211.
- [33] A. K. Dalai and S. K. Jena, "Neutralizing SQL injection attack using server side code modification in web applications," *Secur. Commun. Netw.*, vol. 2017, pp. 1–12, May 2017.
- [34] N. V. Subramanian, N. Saravanan, and V. S. S. Sriram, "Survey on mitigation techniques of virtualization technique," *ARNP J. Eng. Appl. Sci.*, vol. 12, no. 2, pp. 1–12, 2017.
- [35] S. E. Ponta, H. Plate, and A. Sabetta, "Detection, assessment and mitigation of vulnerabilities in open source dependencies," *Empirical Softw. Eng.*, vol. 25, no. 5, pp. 3175–3215, Sep. 2020.
- [36] X. Jin, X. Hu, K. Ying, W. Du, H. Yin, and G. N. Peri, "Code injection attacks on HTML5-based mobile apps: Characterization, detection and mitigation," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2014, pp. 66–77.
- [37] Z. Chen and C. Ji, "Optimal worm-scanning method using vulnerable-host distributions," *Int. J. Secur. Netw.*, vol. 2, nos. 1–2, pp. 71–80, 2007.
- [38] D. Stefan, A. Russo, P. Buiras, A. Levy, J. C. Mitchell, and D. Mazières, "Addressing covert termination and timing channels in concurrent information flow systems," *ACM SIGPLAN Notices*, vol. 47, no. 9, pp. 201–214, Oct. 2012.
- [39] *AWS Blueprint for Ransomware Defense*. Accessed: Jul. 11, 2023. [Online]. Available: <https://d1.awsstatic.com/whitepapers/compliance/AWS-Blueprint-for-Ransomware-Defense.pdf>
- [40] A. S. George and S. Sagayarajan, "Securing cloud application infrastructure: Understanding the penetration testing challenges of IaaS, PaaS, and SaaS environments," *Partners Universal Int. Res. J.*, vol. 2, pp. 24–34, Mar. 2023.
- [41] P. Kumar, "A proposed methodology to mitigate the ransomware attack," in *Recent Trends in Intensive Computing*. Amsterdam, The Netherlands: IOS Press, 2021, pp. 16–21.
- [42] K. S. Vaisla and R. Saini, "Analyzing of zero day attack and its identification techniques," in *Proc. 1st Int. Conf. Adv. Comput. Commun. Eng. (ICACCE)*, 2014, pp. 1–11.
- [43] D. Tripathy, R. Gohil, and T. Halabi, "Detecting SQL injection attacks in cloud SaaS using machine learning," in *Proc. IEEE 6th Int. Conf. Big Data Secur. Cloud (BigDataSecur.) Int. Conf. High Perform. Smart Comput., (HPSC) IEEE Int. Conf. Intell. Data Secur. (IDS)*, May 2020, pp. 145–150.
- [44] S. Soufiane and B. Halima, "SaaS cloud security : Attacks and proposed solutions," *Trans. Mach. Learn. Artif. Intell.*, vol. 5, no. 4, pp. 1–12, Aug. 2017.
- [45] S. Dupont, G. Ginis, M. Malacario, C. Porretti, N. Maunero, C. Ponsard, and P. Massonet, "Incremental common criteria certification processes using DevSecOps practices," in *Proc. IEEE Eur. Symp. Secur. Privacy Workshops (EuroS&PW)*, Sep. 2021, pp. 12–23.



NAGASUNDARI S (Member, IEEE) received the Ph.D. degree from the Faculty of Information and Communication Engineering, Anna University, India. She is currently an Associate Professor with the Department of Computer Science and Engineering and the Centre for Information Security, Forensics and Cyber Resilience, PES University. Her research interests include cloud security and the IoT security. She is an Associate Member of the Institution of Engineers (IEI), India, and a member of ACM and Cloud Security Alliance, Bangalore Chapter.



PRATHAM MANJA received the bachelor's degree in computer science and engineering from PES University. He is currently with IBM Software Labs. He is also associated with the Centre for Information Security, Forensics and Cyber Resilience, PES University. His research interests include software engineering, AI (machine learning and deep learning), cyber security in cloud, and graph analytics.



PRIYANSH MATHUR received the bachelor's degree in computer science and engineering from PES University. He is currently associated with HPE. He is also associated with the Centre for Information Security, Forensics and Cyber Resilience, PES University. His research interests include deep learning, cloud computing and graph neural networks



PRASAD B. HONNAVALLI (Member, IEEE) received the B.E. degree from UVCE, Bangalore University, and the M.B.A. degree from The University of Melbourne, Australia. He is currently a Professor with the Department of Computer Science and Engineering. He is also the Director of the Centre for Information Security, Forensics and Cyber Resilience (C-ISFCR) and the Centre for Internet of Things (C-IoT), PES University, with a focus on security. He is a member of ACM and PMI.

...